Faculdade de Engenharia da Universidade do Porto



Redes de computadores

2º Trabalho Laboratorial

Rede de Computadores - L.EIC | FEUP | 2025/2026

Turma 8

Afonso Saraiva (up202304461) e Hugo Silva (up202307383)

Sumário

Este trabalho, realizado no âmbito da Unidade Curricular de Redes de Computadores, teve como objetivo a implementação de um programa de download utilizando o protocolo FTP e a configuração e utilização de uma rede de computadores.

Introdução

O objetivo principal deste trabalho consistiu no desenvolvimento e teste de um programa de download baseado no protocolo FTP, bem como na configuração de uma rede de computadores de acordo com as especificações presentes no guião fornecido. Esta implementação permitiu a transferência de ficheiros da internet utilizando a infraestrutura de rede configurada ao longo das diversas experiências. O presente relatório encontra-se organizado de forma a documentar sistematicamente todas as etapas realizadas, desde a configuração inicial da rede até à implementação e teste da aplicação de download, incluindo a análise detalhada dos protocolos e mecanismos envolvidos no processo de transferência de dados.

Download program 2
Arquitetura da aplicação 2
Testes e resultados 3
Configuração e análise de redes 3
Experiência 1 - Configurar uma rede IP 3
Experiência 2 - Implementar duas bridges num switch 4
Experiência 3 - Configurar um router em Linux 4
Experiência 4 - Configurar um router comercial com NAT 5
Experiência 5 - DNS 7
Experiência 6 - Ligações TCP 7
Conclusões 8
Referências 9
Anexos 9
Anexo 1 - Código da aplicação de “download.c” 9
Anexo 2 - Guião de configuração (Bancada 12) 13
Starting 13
Experiment 1 13
Experiment 2 14
Experiment 3 14
Experiment 4 14
Experiment 5 15
Anexo 3 - Logs capturados 15

Download program

Arquitetura da aplicação

A aplicação desenvolvida realiza o download de ficheiros através do protocolo FTP. Para a sua implementação, foram consultadas duas normas fundamentais: a RFC959, que define o funcionamento e arquitetura do protocolo FTP, e a RFC1738, que especifica o tratamento das partes constituintes dos endereços URL.

O processamento inicial do URL fornecido como argumento é realizado através da função parseURL, que utiliza manipulação de strings para extrair e organizar numa estrutura de dados todas as informações necessárias para o estabelecimento da ligação. Esta estrutura inclui os seguintes campos: host, correspondente ao nome do servidor onde será criada a comunicação; url_path, que define o caminho completo até ao ficheiro pretendido; file, contendo o nome e extensão do ficheiro a transferir; user e password, utilizados para registo no servidor e que assumem os valores predefinidos "anonymous" e "anonymous" caso não sejam especificados no URL. O endereço IP do servidor é posteriormente obtido através da função gethostbyname, que utiliza o sistema DNS para traduzir o hostname.

A função createSocket é responsável por criar e estabelecer a ligação TCP com o servidor. Esta função recebe como parâmetros o endereço IP do host e a porta a utilizar, cria um socket TCP, configura a estrutura sockaddr_in e estabelece a conexão através da função connect, retornando o descritor do socket criado.

O fluxo de comunicação com o servidor FTP segue uma sequência bem definida de comandos e respostas. Inicialmente, é criado um primeiro socket ligado ao IP do host na porta 21 (porta padrão do FTP) através da função createSocket. Após a receção da mensagem de boas-vindas do servidor (código 220), procede-se à autenticação enviando o comando USER seguido do nome de utilizador e aguardando a resposta 331. De seguida, envia-se o comando PASS com a password, esperando a confirmação 230 que indica login bem-sucedido.

Para estabelecer a transferência de dados, é enviado o comando PASV para colocar o servidor em modo passivo. A resposta (código 227) contém o endereço IP e porta onde o servidor aguarda a segunda conexão, informação que é extraída através de parsing da mensagem recebida. Os valores são obtidos no formato (h1,h2,h3,h4,p1,p2), onde o IP é h1.h2.h3.h4 e a porta é calculada como p1*256+p2. Com estes dados, é criado um segundo socket dedicado à transferência do ficheiro.

O download é iniciado através do comando RETR seguido do caminho do ficheiro. Após a confirmação de início de transferência (código 150 ou 125), os dados são recebidos através do segundo socket e escritos diretamente num ficheiro local utilizando as funções read e fwrite. A transferência continua em ciclo até que não existam mais dados a receber. Após a conclusão, o servidor envia o código 226 confirmando o sucesso da operação.

Finalmente, ambos os sockets são encerrados de forma ordenada: primeiro o socket de dados após o término da transferência, e posteriormente o socket de controlo após o envio do comando QUIT e receção da resposta 221 do servidor.

Cada operação de comunicação com o servidor inclui validação do código de resposta recebido. Caso o código não corresponda ao esperado em qualquer etapa do processo, a aplicação é terminada com uma mensagem de erro que identifica claramente o ponto de falha, garantindo robustez no tratamento de situações anómalas.

Testes e resultados

De modo a testar o correto funcionamento da aplicação de *download*, foram realizados vários testes com transferências de ficheiros de diferentes tamanhos. Foram também testadas as várias camadas de validação de input, através URLs errados em diversos campos, *resources* não existentes e modo anónimo e não anónimo. Verificamos que em todas as situações apresentadas o programa mostrou-se robusto e eficiente.

Configuração e análise de redes

Experiência 1 - Configurar uma rede IP

Esta experiência consistiu na configuração de endereços IP em dois computadores distintos, o Tux123 e Tux124, interligados através de um switch. Subsequentemente, pretendeu-se examinar o funcionamento do protocolo ARP e observar o comportamento da conectividade quando se removem registos dos IPs configurados das tabelas ARP.

Para a realização desta experiência, conectou-se a interface E1 de cada Tux ao switch e recorreu-se ao comando ifconfig para atribuir os endereços IP. Os endereços MAC e IP de cada computador foram consultados nas tabelas ARP geradas após confirmar a conectividade entre ambos os computadores através do comando ping. No Tux123 foi atribuído o IP 172.16.120.1/24, correspondente ao MAC 00:75:0c:c2:3c:7b, enquanto no Tux124 o IP configurado foi 172.16.120.254/24 com o MAC 00:75:0c:c2:3c:75. Ao examinar os pacotes transmitidos após a execução deste comando, observa-se a existência de pacotes ARP e ICMP.

O protocolo ARP estabelece a correspondência entre um endereço IP de uma máquina e o respetivo endereço MAC, realizando portanto a ligação entre a camada de rede e a camada de ligação de dados, sendo desta forma essencial para o estabelecimento de qualquer comunicação. Durante uma requisição ARP, são transmitidos em broadcast os endereços IP: o da máquina de destino e o da máquina de origem. Em resposta, o destinatário transmite um pacote do mesmo protocolo contendo o seu endereço MAC. Esta correspondência fica armazenada numa entrada da tabela ARP de ambas as máquinas.

Observou-se que ao eliminar as entradas das tabelas ARP, ocorre uma nova troca de pacotes ARP no início da comunicação para restabelecer as associações entre endereços IP e MAC que foram removidas. Esta sequência de mensagens ficou evidenciada nas primeiras linhas da captura de pacotes.

O protocolo ICMP é empregue para a troca de mensagens de controlo, sinalizando sucesso ou falhas durante a comunicação com outro endereço IP. No contexto específico desta experiência, foram identificados pacotes transportando mensagens de request e reply.

O registo da captura de pacotes durante a execução do comando ping encontra-se disponível no anexo 3 exp1, e os comandos utilizados no anexo 2 exp1.

A interface loopback não foi utilizada no decorrer desta experiência, uma vez que toda a comunicação analisada ocorreu entre máquinas distintas ligadas através de um switch. No entanto, a interface loopback consiste numa interface virtual que permite a uma máquina comunicar consigo própria. Esta interface é amplamente utilizada para fins de teste, diagnóstico e comunicação interna entre processos.

Experiência 2 - Implementar duas *bridges* num switch

Esta experiência consistiu na criação de duas LANs (Local Area Network) distintas, uma contendo os Tux123 e Tux124, e outra exclusivamente com o Tux122, recorrendo à implementação de 2 bridges no switch.

Numa fase inicial, foi necessário replicar a configuração de rede da experiência 1 e atribuir o endereço IP 172.16.121.1/24 ao Tux122, correspondente ao MAC 00:75:0c:c2:31:73. Para estabelecer as bridges foi imprescindível configurar o switch, procedimento que foi realizado conectando a consola deste à porta série do Tux122 através do GTK. Durante a configuração, foram estabelecidas as bridges 120 e 121 (utilizando o comando /interface bridge add), foram desassociadas as portas às quais as interfaces ligadas a cada um dos Tuxes estavam conectadas por omissão (através do comando /interface bridge port remove), e posteriormente foram integradas as interfaces dos Tux123 e Tux124 na bridge120, enquanto a interface do Tux122 foi adicionada à bridge121 (recorrendo ao comando /interface bridge port add).

Nesta experiência existiam dois domínios de broadcast distintos, correspondentes às duas bridges/LANs implementadas. Constatou-se que ao realizar um broadcast a partir do Tux123, o Tux124 era atingido mas o Tux122 não, dado que apenas o primeiro se encontra na mesma LAN que o Tux123. Executando o mesmo procedimento no Tux122, confirmou-se que este não estabelece comunicação com nenhum outro computador, uma vez que se encontra numa LAN isolada. Esta conclusão foi obtida através da análise dos pacotes ICMP capturados nos diferentes computadores: na primeira situação verificaram-se estados alternados de request/reply, ao passo que na segunda apenas se registaram packets de request sem qualquer resposta correspondente.

Os registos da captura de pacotes durante a execução do comando ping nos Tux123 e Tux122 encontram-se disponíveis no anexo 3 exp2, respetivamente, e os comandos empregues para a configuração no anexo 2 exp 2.

Experiência 3 - Configurar um router em Linux

Nesta experiência configuramos o Tux124 como router para permitir a comunicação entre o Tux123 e o Tux122 através das duas subredes 172.16.120.0/24 e 172.16.121.0/24.

Nesta experiência, partindo da configuração de rede final da experiência anterior, configurámos o Tux124 como router Linux (ligando-o ao switch e atribuindo-o à bridge correta), ativando o IP forwarding. As tabelas de encaminhamento das três máquinas tux foram analisadas utilizando o comando route -n. Cada tux continha uma rota para a sua própria rede diretamente conectada, o que permite comunicação local sem necessidade de gateway. Adicionalmente, o Tux123 e o Tux122 tinham cada um uma rota estática apontando para o Tux124 como gateway para a outra rede remota. Isto significa que qualquer pacote destinado a um endereço IP fora da subrede local é encaminhado para o Tux124, que é responsável por encaminhar o tráfego entre as duas subredes 172.16.120.0/24 e 172.16.121.0/24. Estas rotas definem a lógica de encaminhamento da rede e garantem a entrega correta de pacotes através de diferentes subredes.

Cada entrada na tabela de encaminhamento (routing) contém informação essencial

necessária para o encaminhamento de pacotes, incluindo a rede de destino, o gateway, a máscara de rede (Genmask), as flags de encaminhamento, a métrica, a contagem de referência (Ref), a contagem de uso, e a interface de saída (Iface). Em conjunto, estes campos permitem ao sistema operativo determinar se um pacote deve ser entregue diretamente na rede local ou encaminhado para um router para entrega posterior.

Durante as capturas de pacotes, foram observadas várias mensagens ARP. Quando o Tux123 iniciou comunicação com o Tux122, pedidos ARP foram enviados em broadcast para resolver o endereço MAC do gateway, não do host de destino final. O Tux124 respondeu com o endereço MAC da interface conectada à subrede do emissor. Este comportamento ocorre porque os hosts comunicam na Camada de Ligação de Dados apenas dentro da sua rede local e, portanto, necessitam do endereço MAC do router para alcançar redes remotas. Como resultado, o endereço MAC associado ao IP do gateway é utilizado como endereço MAC de destino.

Os pacotes ICMP observados durante a experiência consistiram principalmente em mensagens Echo Request e Echo Reply geradas pelo comando ping. Estes pacotes foram utilizados para testar a conectividade entre hosts localizados em diferentes subredes e para verificar que o encaminhamento estava a funcionar corretamente. Os ICMP Echo Requests originaram-se no Tux123 e foram encaminhados pelo Tux124 para o Tux122, enquanto os Echo Replies seguiram o caminho inverso de volta à origem.

A análise dos pacotes ICMP mostrou que os endereços IP permaneceram inalterados da origem ao destino, correspondendo aos IPs do Tux123 e Tux122. Contudo, os endereços MAC alteraram-se em cada salto. Quando os pacotes viajaram do Tux123 para o Tux124, o endereço MAC de destino era o MAC da interface do Tux124 nessa subrede. Após o encaminhamento, o Tux124 reencaminhou o pacote em direção ao Tux122 utilizando o endereço MAC do Tux122 como destino. Isto confirma que o encaminhamento opera na camada IP, enquanto os endereços MAC são apenas relevantes dentro de um único segmento de rede local e são reescritos em cada salto.

Adicionalmente, foram removidas as entradas das tabelas ARP em todos os Tuxes utilizando o comando arp -d [endereço_ip], garantindo a observação completa do processo de resolução de endereços durante a comunicação inter-LAN.

Os registos da captura de pacotes durante a execução do comando ping no Tux123 encontram-se disponíveis no anexo 3 exp3 e os comandos empregues para a configuração no anexo 2 exp3.

Experiência 4 - Configurar um router comercial com NAT

Nesta experiência foi introduzido um router comercial (RC) na rede previamente configurada na Experiência 3, tendo como finalidade disponibilizar acesso à Internet através da implementação de NAT. A rede era composta por duas subredes internas, 172.16.120.0/24 e 172.16.121.0/24, interligadas pelo Tux124, e uma rede externa 172.16.1.0/24 com ligação à Internet. A interface ether1 do router comercial foi ligada à rede do laboratório com NAT ativado por predefinição, enquanto a ether2 foi conectada à bridge121. Os endereços IP das interfaces do router foram definidos através da consola série do router, e foram adicionadas rotas estáticas onde necessário para assegurar conectividade completa entre todas as redes.

As rotas estáticas no router comercial foram estabelecidas especificando a rede de

destino, a respetiva máscara de rede, e o endereço IP do próximo salto que conduz a essa rede. Neste caso, foi adicionada uma rota no RC apontando para a rede interna 172.16.120.0/24 via Tux124, permitindo que o tráfego de retorno proveniente da Internet alcance as subredes internas. De forma semelhante, o Tux123, Tux124 e Tux122 foram configurados com rotas para alcançar a rede 172.16.1.0/24 através do RC.

Durante as experiências envolvendo ICMP redirects, foram observados diferentes percursos de pacotes dependendo de se a aceitação de ICMP redirect estava ativada ou desativada. Quando os ICMP redirects foram desativados no Tux122 e o RC foi configurado como gateway para alcançar 172.16.120.0/24, os pacotes do Tux122 para o Tux123 seguiram um percurso mais longo, passando primeiro pelo RC, depois pelo Tux124, e finalmente chegando ao Tux123. Isto ocorreu porque o Tux122 seguiu estritamente a sua tabela de encaminhamento e não aceitou sugestões de otimização. Quando os ICMP redirects foram ativados, o Tux122 recebeu mensagens ICMP redirect indicando que o Tux124 era um gateway melhor para alcançar a rede de destino. Como resultado, os pacotes subsequentes foram enviados diretamente para o Tux124, reduzindo o número de saltos e melhorando a eficiência do encaminhamento. O traceroute confirmou estas diferenças ao mostrar menos saltos intermédios quando os redirects eram aceites.

O NAT foi configurado no router comercial através da ativação da tradução de endereços de origem na interface externa ligada à Internet. Esta configuração permite que pacotes originários das redes internas privadas sejam traduzidos num único endereço IP público ao sair do router. O NAT funciona substituindo o endereço IP de origem dos pacotes de saída pelo IP público do router e mantendo uma tabela de tradução que mapeia endereços e portas privados internos para externos. Quando as respostas chegam da Internet, o router utiliza esta tabela para traduzir o endereço de destino de volta para o host interno original.

Com o NAT ativado, o Tux123 conseguiu efetuar ping com sucesso ao servidor FTP em 172.16.1.10. Os pacotes saíram do Tux123 com um endereço IP de origem privado, foram traduzidos pelo RC num endereço público, e encaminhados para a rede externa. As respostas do servidor FTP foram corretamente traduzidas de volta e entregues ao Tux123, confirmando o correto funcionamento do NAT.

Quando a funcionalidade NAT foi desativada no router comercial, o Tux123 deixou de conseguir alcançar o servidor FTP. Embora os pacotes ICMP Echo Request chegassem ao servidor, este tentava responder diretamente ao endereço IP privado do Tux123, que não é encaminhável na rede externa. Uma vez que o RC já não estava a traduzir endereços, não possuía informação de mapeamento para reencaminhar as respostas de volta ao host interno, resultando numa perda de conectividade. Este comportamento demonstra que o NAT é essencial para permitir a comunicação entre redes privadas e a Internet pública e também evidencia o seu papel na conservação de endereços IP públicos e na garantia de encaminhamento correto de pacotes.

Os comandos empregues para a configuração encontram-se disponíveis no anexo 2 exp4, e os registos da captura de pacotes no anexo 3 exp4.

Experiência 5 - DNS

Para esta experiência utilizou-se a rede configurada ao longo das experiências anteriores com o objetivo de configurar o serviço DNS de modo a permitir o acesso a websites da internet através do seu nome de domínio, dentro da rede criada. O correto funcionamento foi verificado através do comando ping ao domínio google.com.

A configuração do serviço DNS revelou-se bastante simples, sendo apenas necessário modificar o conteúdo do ficheiro /etc/resolv.conf em cada Tux. Esta modificação consistiu em adicionar a linha 'nameserver 172.16.1.1', que corresponde ao endereço IP do router presente no laboratório. Este router possui acesso à rede externa e funciona como servidor DNS para a rede local, permitindo assim a resolução de nomes de domínio.

Durante a análise dos logs capturados, observou-se um comportamento importante do protocolo DNS na rede: a troca de pacotes DNS ocorre antes de qualquer outro protocolo de comunicação. Esta precedência é necessária porque o DNS realiza a tradução de um nome de domínio para o seu endereço IP correspondente, informação essencial que será posteriormente utilizada por todos os outros protocolos de rede. Sem esta resolução prévia de nomes, os protocolos de camadas superiores não teriam conhecimento dos endereços IP necessários para estabelecer comunicação com os servidores remotos.

Os comandos utilizados para a configuração estão documentados no anexo 2 exp5, e os logs completos da captura de pacotes podem ser consultados no anexo 3 exp5.

Experiência 6 - Ligações TCP

Nesta experiência, além de utilizarmos toda a rede configurada ao longo dos passos anteriores, recorremos à aplicação de download que desenvolvemos para avaliar o envio e receção de pacotes, a sua constituição interna e os mecanismos de controlo de fluxo e congestionamento característicos das ligações TCP.

Numa primeira fase, compilámos o downloader no Tux123 e realizámos o download de um ficheiro com alguns megabytes. Durante a captura de pacotes, observou-se uma sequência bem definida de eventos: inicialmente ocorreu a transferência de pacotes DNS para tradução do nome do servidor no endereço IP correspondente, seguida de pacotes FTP SYN-ACK request/response para estabelecer o handshake entre o cliente e o servidor. Posteriormente, foram transmitidos pacotes FTP Data com números de sequência incrementais para receção das tramas de informação. A ligação terminou com o envio do pacote FTP FIN-ACK, sinalizando o encerramento da comunicação.

A aplicação FTP utilizada abre duas ligações TCP (Transmission Control Protocol) distintas: uma dedicada ao envio de comandos de controlo ao servidor e outra para a receção do ficheiro selecionado. Todas as ligações TCP utilizam ARQ, são orientadas à conexão e garantem a transferência fiável de dados através da incorporação de dois mecanismos de controlo fundamentais: o controlo de fluxo e o controlo de congestionamento.

O mecanismo ARQ (Automatic Repeat Request) é utilizado nas ligações TCP para controlo de erros, recorrendo a mensagens ACK (acknowledge) para indicar a correta receção de cada pacote e a timeouts para avaliar o tempo de receção dos mesmos. Sempre que um timeout é ultrapassado, o sistema considera que o pacote foi perdido e procede à sua

retransmissão através da rede.

O mecanismo de controlo de fluxo permite ao recetor regular o seu próprio ritmo de receção de pacotes. Uma vez que os buffers do lado do recetor e do emissor são finitos, estes não podem receber mais informação do que a sua capacidade, pois caso contrário ocorreria perda de pacotes na rede, desencadeando retransmissões desnecessárias. Para evitar esta situação, cada pacote transporta um campo window size em bytes, permitindo ao emissor avaliar a quantidade de dados que ainda pode enviar sem comprometer a viabilidade da transmissão. Durante a análise dos pacotes transferidos ao longo desta experiência, tornou-se evidente que o campo window size apresentava valores inferiores quando existiam picos de tráfego na rede, demonstrando o funcionamento dinâmico deste mecanismo.

O controlo de congestionamento é efetuado pelos emissores de dados e baseia-se na utilização do método Selective Repeat, que consiste no envio de múltiplos pacotes pela rede sem aguardar pelos respetivos ACK. Uma rede considera-se congestionada a partir do momento em que ocorre perda de pacotes. Para determinar o limite da capacidade da rede durante a transferência, o emissor transmite vários pacotes simultaneamente seguindo uma de duas metodologias até detetar perda de pacotes: Additive Increase, onde na transferência seguinte envia mais um pacote do que na transferência anterior, ou Slow Start, semelhante à anterior mas com incrementos exponenciais na base dois. A perda de um pacote pode ser detetada através de timeout ou de três ACK duplicados consecutivos, provocando uma redução drástica do número de pacotes transferidos na transmissão seguinte para valores próximos de metade do valor que despoletou a perda.

No gráfico obtido durante esta primeira fase, onde se analisou a velocidade dos pacotes encaminhados pela rede em função do tempo, ficou claramente visível a influência deste mecanismo característico das ligações TCP. Efetivamente, após períodos de aumento significativo no número de pacotes transferidos, que correspondem a Additive Increase ou Slow Start consoante os declives sejam menos ou mais acentuados respetivamente, observa-se um decréscimo abrupto desse valor para aproximadamente metade do valor anterior.

Numa segunda fase, iniciou-se um download de um ficheiro com alguns megabytes no Tux123 e, logo a seguir, outro download no Tux122. É possível observar no gráfico da velocidade de transferência que esta diminui para cerca de metade no Tux123 quando o segundo download é iniciado. Este comportamento deve-se à ação do controlo de congestionamento por parte dos dois computadores na rede, que converge rapidamente para valores estáveis ao longo da transferência mútua, demonstrando a capacidade do protocolo TCP em distribuir equitativamente a largura de banda disponível entre múltiplas conexões.

Os logs da captura de pacotes encontram-se disponíveis no anexo3 exp6.

Conclusões

Neste projeto, consolidamos o nosso conhecimento acerca dos protocolos envolvidos na transferência de dados através de redes de computadores, nomeadamente como estes se propagam dentro *network layer* e a sua passagem para a camada de *link layer*.

Referências

1. RFCs relevantes fornecidos no moodle.
2. Beej's Guide to Network Programming Using Internet Sockets
3. Mikrotik RouterOS Wiki

Anexos

Anexo 1 - Código da aplicação de "download.c"

```c
#include <stdio.h>
#include <stdlib.h>
#include <netdb.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>
#include "download.h"

int parseURL(char *link, url_info *info) {
    char *link_copy = link;

    strcpy(info->user, "anonymous");
    strcpy(info->password, "anonymous@");

    if (strncmp(link_copy, "ftp://", 6) != 0) {
        printf("URL invalid. Must start with 'ftp://'\n");
        return -1;
    }

    link_copy += 6;

    char *at_sign = strchr(link_copy, '@');
    char *slash = strchr(link_copy, '/');

    if (slash == NULL) {
        printf("URL invalid. Missing '/' after host.\n");
        return -1;
    }

    if (at_sign != NULL && at_sign < slash) {
        char *colon = strchr(link_copy, ':');

        if (colon == NULL || colon > at_sign) {
            printf("Invalid 'user:password' format.\n");
            return -1;
        }

        size_t user_length = colon - link_copy;
        strncpy(info->user, link_copy, user_length);
        info->user[user_length] = '\0';

        size_t password_length = at_sign - colon - 1;
        strncpy(info->password, colon + 1, password_length);
        info->password[password_length] = '\0';

        link_copy = at_sign + 1;

        char *slash = strchr(link_copy, '/');

        if (slash == NULL) {
            printf("URL invalid. Missing '/' after host.\n");
```

```
            return -1;
        }
    }

    size_t host_length = slash - link_copy;
    strncpy(info->host, link_copy, host_length);
    info->host[host_length] = '\0';

    strcpy(info->url_path, slash + 1);

    char *last_slash = strrchr(info->url_path, '/');
    if (last_slash != NULL) {
        strcpy(info->file, last_slash + 1);
    } else {
        strcpy(info->file, info->url_path);
    }

    return 0;
}

int createSocket(char *host, int port) {
    int sockfd;
    struct sockaddr_in server_addr;

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = inet_addr(host);
    server_addr.sin_port = htons(port);

    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("socket()");
        exit(-1);
    }
    if (connect(sockfd, (struct sockaddr *) &server_addr, sizeof(server_addr)) < 0) {
        perror("connect()");
        exit(-1);
    }

    return sockfd;
}

int main(int argc, char *argv[]) {

    char buf[500];

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <ftp_url>\n", argv[0]);
        exit(-1);
    }

    url_info info;

    memset(&info, 0, sizeof(info));

    if (parseURL(argv[1], &info) != 0) {
        printf("Failed to parse URL.\n");
        exit(-1);
    }

    printf("User: %s\n", info.user);
    printf("Password: %s\n", info.password);
    printf("Host: %s\n", info.host);
    printf("URL Path: %s\n", info.url_path);
    printf("File: %s\n", info.file);

    struct hostent *h;
    h = gethostbyname(info.host);
    if (h == NULL) {
        printf("DNS Error\n");
```

```
    exit(-1);
}

char *host_ip = inet_ntoa(*((struct in_addr *) h->h_addr));
printf("IP: %s\n", host_ip);

int sock = createSocket(host_ip, 21);
printf("Socket created and connected to %s on port 21\n", host_ip);

usleep(100000);
size_t read_bytes = read(sock, buf, 500);
if (read_bytes > 0) {
    buf[read_bytes] = '\0';
    printf("Received (%ld bytes): %s", read_bytes, buf);
} else {
    printf("Failed to read from socket.\n");
    exit(-1);
}

if (strncmp(buf, "220", 3) != 0) {
    printf("FTP server not ready to connect.\n");
    exit(-1);
}

char user_cmd[100];
sprintf(user_cmd, "USER %s\r\n", info.user);
write(sock, user_cmd, strlen(user_cmd));
printf("Sent: %s", user_cmd);

usleep(100000);
size_t user_bytes = read(sock, buf, sizeof(buf) - 1);
buf[user_bytes] = '\0';
printf("Received (%ld bytes): %s", user_bytes, buf);

if (strncmp(buf, "331", 3) != 0) {
    printf("Username not accepted.\n");
    exit(-1);
}

char password_cmd[100];
sprintf(password_cmd, "PASS %s\r\n", info.password);
write(sock, password_cmd, strlen(password_cmd));
printf("Sent: %s", password_cmd);

usleep(100000);
size_t password_bytes = read(sock, buf, sizeof(buf) - 1);
buf[password_bytes] = '\0';
printf("Received (%ld bytes): %s", password_bytes, buf);

if (strncmp(buf, "230", 3) != 0) {
    printf("Password not accepted.\n");
    exit(-1);
}

char passive_cmd[100] = "PASV\r\n";
write(sock, passive_cmd, strlen(passive_cmd));
printf("Sent: %s", passive_cmd);

usleep(100000);
size_t passive_bytes = read(sock, buf, sizeof(buf) - 1);
buf[passive_bytes] = '\0';
printf("Received (%ld bytes): %s", passive_bytes, buf);

if (strncmp(buf, "227", 3) != 0) {
    printf("PASV command failed.\n");
    exit(-1);
}

int h1 = -1, h2 = -1, h3 = -1, h4 = -1, p1 = -1, p2 = -1;
```

```
int count = 0, num = 0, inside = 0;

for (int i = 0; buf[i] != '\0'; i++) {
    char c = buf[i];

    if (c == '(') {
        inside = 1;
        continue;
    }
    if (c == ')') {
        p2 = num;
        break;
    }

    if (!inside) continue;

    if (isdigit(c)) {
        num = num * 10 + (c - '0');
    } else if (c == ',') {
        count++;
        switch (count) {
            case 1: h1 = num; break;
            case 2: h2 = num; break;
            case 3: h3 = num; break;
            case 4: h4 = num; break;
            case 5: p1 = num; break;
            default: break;
        }
        num = 0;
    }
}

char server_ip[32];
sprintf(server_ip, "%d.%d.%d.%d", h1, h2, h3, h4);
int server_port = p1 * 256 + p2;

int data_sock = createSocket(server_ip, server_port);

char retrieve_cmd[200];
sprintf(retrieve_cmd, "RETR %s\r\n", info.url_path);
write(sock, retrieve_cmd, strlen(retrieve_cmd));
printf("Sent: %s", retrieve_cmd);

size_t retrieve_bytes = read(sock, buf, sizeof(buf) - 1);
buf[retrieve_bytes] = '\0';
printf("Received (%ld bytes): %s", retrieve_bytes, buf);

if (strncmp(buf, "150", 3) != 0 && strncmp(buf, "125", 3) != 0) {
    printf("RETR command failed.\n");
    exit(-1);
}

FILE *file = fopen(info.file, "wb");
if (file == NULL) {
    printf("Failed to open file for writing.\n");
    exit(-1);
}

int n = 0;
while ((n = read(data_sock, buf, sizeof(buf))) > 0) {
    if (fwrite(buf, 1, n, file) != n) {
        printf("Failed to write to file.\n");
        fclose(file);
        exit(-1);
    }
}

fclose(file);
```

```
    if (close(data_sock) < 0) {
        printf("Failed to close data socket.\n");
        exit(-1);
    }

    size_t exit_bytes = read(sock, buf, sizeof(buf) - 1);
    buf[exit_bytes] = '\0';
    printf("Received (%ld bytes): %s", exit_bytes, buf);

    if (strncmp(buf, "226", 3) != 0) {
        printf("File transfer not completed successfully.\n");
        exit(-1);
    }

    char quit_cmd[100] = "QUIT\r\n";
    write(sock, quit_cmd, strlen(quit_cmd));
    printf("Sent: %s", quit_cmd);

    size_t quit_bytes = read(sock, buf, sizeof(buf) - 1);
    buf[quit_bytes] = '\0';
    printf("Received (%ld bytes): %s", quit_bytes, buf);

    if (strncmp(buf, "221", 3) != 0) {
        printf("QUIT command failed.\n");
        exit(-1);
    }

    if (close(sock) < 0) {
        printf("Failed to close control socket.\n");
        exit(-1);
    }

    return 0;
}
```

Anexo 2 - Guião de configuração (Bancada 12)

Starting

sudo su
route: system reset-config
mudar para Switch - (cabos)

GTK
Tux2:system reset-config
sysctl net.ipv4.ip_forward=1
sysctl nte.ipv4.icmp_echo_ignore_broadcasts=0

Experiment 1

Tux4: sudo ifconfig if_e1 172.16.110.254/24
Tux3: sudo ifconfig if_e1 172.16.110.1/24
// Tux3 MAC address: 75:0c:c2:3c:7b
// Tux4 MAC address: 75:0c:c2:3c:75

Clear arp table using sudo arp -d [ip address]
Example:
Tux12.3: sudo arp -d 172.16.110.254
Tux12.3: sudo arp -d 10.227.20.2
Tux12.3: sudo arp -d 10.227.20.223
Tux12.3: sudo arp -d 10.227.20.254

Tux12.3: sudo arp -d 10.227.20.3

Experiment 2

Tux2: sudo ifconfig if_e1 172.16.121.1/24
Tux2 MAC address: 75:0c:c2:31:73

GTK:
Tux2: /interface bridge add name=bridge120
Tux2: /interface bridge add name=bridge121

-- Meter cabos connectados, o do Tux2 ao 2, o Tux3 ao 3 e o Tux4 ao 4
Tux2: /interface bridge port remove [find interface=ether2]
Tux2: /interface bridge port remove [find interface=ether3]
Tux2: /interface bridge port remove [find interface=ether4]

-- Tux 2 Separado do Tux3 e Tux4
Tux2: /interface bridge port add bridge=bridge121 interface=ether2
Tux2: /interface bridge port add bridge=bridge120 interface=ether3
Tux2: /interface bridge port add bridge=bridge120 interface=ether4

// Depois fizemos pings para verificar q o tux 3 comunica com o 4, mas não com o 2 visto q estão em bridges diferentes.

Experiment 3

Tux4: sudo ifconfig if_e2 172.16.121.253/24
Tux2: /interface bridge port remove [find interface=ether24] – liga tux4 e2 ao 24 do switch
Tux2: /interface bridge port add bridge=bridge121 interface=ether24

Tux4 MAC address: 75:0c:2c:3c:75 -> if_e1
Tux4 MAC address: 75:0c:c2:10:6b -> if_e2

Tux3: route add -net 172.16.121.0/24 gw 172.16.120.254
Tux2: route add -net 172.16.120.0/24 gw 172.16.121.253

Clear arp tables on all tuxes using arp -d [ip_address]
Example:
Tux2: arp -d 172.16.121.253
Tux3: arp -d 172.16.120.254
Tux4: arp -d 172.16.121.1
arp -d 172.16.120.1

Experiment 4

GTK:
Tux2 (Switch console):
/interface bridge port remove [find interface=ether9]
/interface bridge port add bridge=bridge121 interface=ether9
Tux2 (Router console):
/ip address add address=172.16.1.121/24 interface=ether1
/ip address add address=172.16.121.254/24 interface=ether2

Tux2: route -n
Tux2: route add -net 172.16.1.0/24 gw 172.16.121.254 – conecta ao FTP Server por gw do router
Tux3: route -n (estavam lá as duas já, senão adicionar)

Tux4: route -n
Tux4: route add -net 172.16.1.0/24 gw 172.16.121.254

GTK:
Tux2 (router console):
/ip route add dst-address=172.16.110.0/24 gateway=172.16.121.253

Tux2:
sysctl net.ipv4.conf.if_e1.accept_redirects=0
sysctl net.ipv4.conf.all.accept_redirects=0

Tux2:
route -n
route del -net 172.16.120.0/24 gw 172.16.121.253
route add -net 172.16.120.0/24 gw 172.16.121.254
// O Tux2 fala com o Rc. Dps o Rc fala com o Tux4. E o Tux4 pede a mensagem ao Tux2.

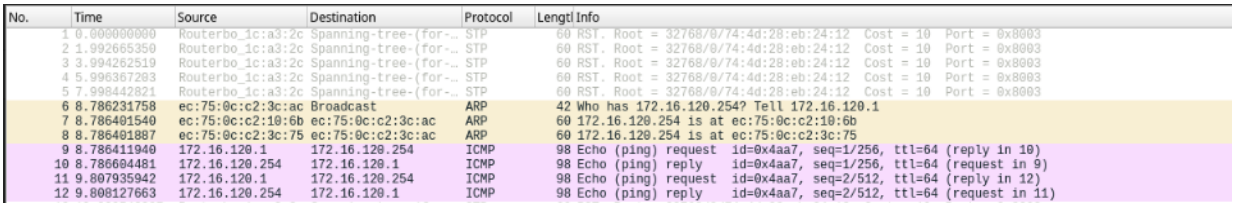
Tux2: traceroute tux123 //10.227.2.113 – 30hops max, 60byte packets - 1.3 …
Tux2:
route -n
route del -net 172.16.120.0/24 gw 172.16.121.254
route add -net 172.16.120.0/24 gw 172.16.121.253
Tux2: traceroute tux123 //10.227.2.123 – 30hops max, 60byte packets - 0.5 …
Tux2: sysctl net.ipv4.conf.if_e1.accept_redirects=1
sysctl net.ipv4.conf.all.accept_redirects=1
Tux2: route -n
route del -net 172.16.120.0/24 gw 172.16.121.253
route add -net 172.16.120.0/24 gw 172.16.121.254
Tux2 (GTK):
/ip firewall nat disable 0

Experiment 5

Tux52, Tux53, Tux54:
ping google.com (responde depois de configurar o DNS)

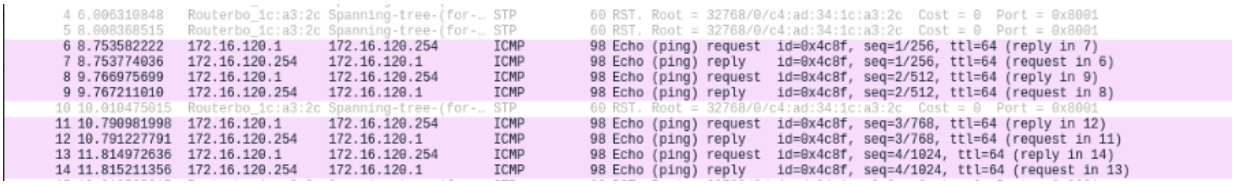
Anexo 3 - Logs capturados

Exp1 - Ping do Tux123 ao Tux124



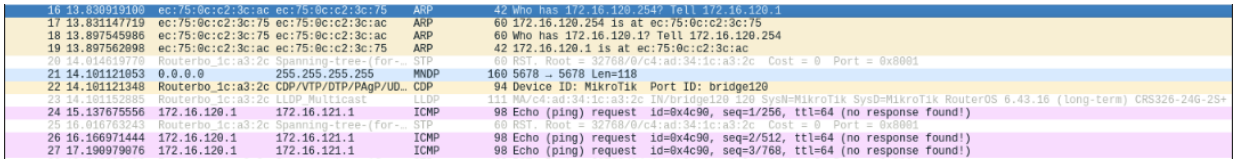
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:eb:24:12 Cost = 10 Port = 0x8003
2	1.992665350	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:eb:24:12 Cost = 10 Port = 0x8003
3	3.994262519	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:eb:24:12 Cost = 10 Port = 0x8003
4	5.996367203	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:eb:24:12 Cost = 10 Port = 0x8003
5	7.998442821	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/74:4d:28:eb:24:12 Cost = 10 Port = 0x8003
6	8.786231758	ec:75:0c:c2:3c:ac	Broadcast	ARP	42	Who has 172.16.120.254? Tell 172.16.120.1
7	8.786401540	ec:75:0c:c2:10:6b	ec:75:0c:c2:3c:ac	ARP	60	172.16.120.254 is at ec:75:0c:c2:10:6b
8	8.786401887	ec:75:0c:c2:3c:75	ec:75:0c:c2:3c:ac	ARP	60	172.16.120.254 is at ec:75:0c:c2:3c:75
9	8.786411940	172.16.120.1	172.16.120.254	ICMP	98	Echo (ping) request id=0x4aa7, seq=1/256, ttl=64 (reply in 10)
10	8.786604481	172.16.120.254	172.16.120.1	ICMP	98	Echo (ping) reply id=0x4aa7, seq=1/256, ttl=64 (request in 9)
11	9.807935942	172.16.120.1	172.16.120.254	ICMP	98	Echo (ping) request id=0x4aa7, seq=2/512, ttl=64 (reply in 12)
12	9.808127663	172.16.120.254	172.16.120.1	ICMP	98	Echo (ping) reply id=0x4aa7, seq=2/512, ttl=64 (request in 11)

Exp2 - Ping do Tux123 ao Tux144



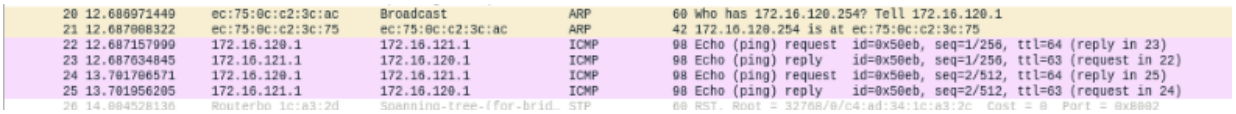
4	6.006310848	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:a3:2c Cost = 0 Port = 0x8001
5	8.008368515	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:a3:2c Cost = 0 Port = 0x8001
6	8.753582222	172.16.120.1	172.16.120.254	ICMP	98	Echo (ping) request id=0x4c8f, seq=1/256, ttl=64 (reply in 7)
7	8.753774036	172.16.120.254	172.16.120.1	ICMP	98	Echo (ping) reply id=0x4c8f, seq=1/256, ttl=64 (request in 6)
8	9.766975699	172.16.120.1	172.16.120.254	ICMP	98	Echo (ping) request id=0x4c8f, seq=2/512, ttl=64 (reply in 9)
9	9.767211010	172.16.120.254	172.16.120.1	ICMP	98	Echo (ping) reply id=0x4c8f, seq=2/512, ttl=64 (request in 8)
10	10.010475015	Routerbo_1c:a3:2c	Spanning-tree-(for-…	STP	60	RST. Root = 32768/0/c4:ad:34:1c:a3:2c Cost = 0 Port = 0x8001
11	10.790981998	172.16.120.1	172.16.120.254	ICMP	98	Echo (ping) request id=0x4c8f, seq=3/768, ttl=64 (reply in 12)
12	10.791227791	172.16.120.254	172.16.120.1	ICMP	98	Echo (ping) reply id=0x4c8f, seq=3/768, ttl=64 (request in 11)
13	11.814972636	172.16.120.1	172.16.120.254	ICMP	98	Echo (ping) request id=0x4c8f, seq=4/1024, ttl=64 (reply in 14)
14	11.815211356	172.16.120.254	172.16.120.1	ICMP	98	Echo (ping) reply id=0x4c8f, seq=4/1024, ttl=64 (request in 13)

Exp2 - Ping do Tux123 ao Tux122 (sem resposta)



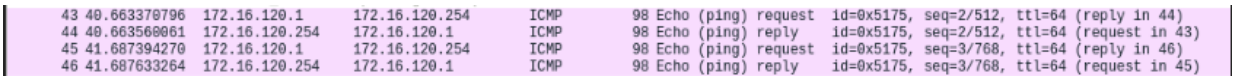
```
16 13.830919100  ec:75:0c:c2:3c:ac ec:75:0c:c2:3c:75  ARP    42 Who has 172.16.120.254? Tell 172.16.120.1
17 13.831147719  ec:75:0c:c2:3c:75 ec:75:0c:c2:3c:ac  ARP    60 172.16.120.254 is at ec:75:0c:c2:3c:75
18 13.897545986  ec:75:0c:c2:3c:75 ec:75:0c:c2:3c:ac  ARP    60 Who has 172.16.120.1? Tell 172.16.120.254
19 13.897562698  ec:75:0c:c2:3c:ac ec:75:0c:c2:3c:75  ARP    42 172.16.120.1 is at ec:75:0c:c2:3c:ac
20 14.014619778  Routerbo_1c:a3:2c Spanning-tree-(for-… STP    60 RST. Root = 32768/0/c4:ad:34:1c:a3:2c  Cost = 0  Port = 0x8001
21 14.101121053  0.0.0.0           255.255.255.255     MNDP   160 5678 → 5678 Len=118
22 14.101121348  Routerbo_1c:a3:2c CDP/VTP/DTP/PAgP/UD… CDP    94 Device ID: MikroTik  Port ID: bridge120
23 14.101152885  Routerbo_1c:a3:2c LLDP Multicast       LLDP   111 MA/c4:ad:34:1c:a3:2c IN/bridge120 120 SysN=MikroTik SysD=MikroTik RouterOS 6.43.16 (long-term) CRS326-24G-2S+
24 15.137675556  172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) request  id=0x4c90, seq=1/256, ttl=64 (no response found!)
25 16.016763243  Routerbo_1c:a3:2c Spanning-tree-(for-… STP    60 RST. Root = 32768/0/c4:ad:34:1c:a3:2c  Cost = 0  Port = 0x8001
26 16.166971444  172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) request  id=0x4c90, seq=2/512, ttl=64 (no response found!)
27 17.190979076  172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) request  id=0x4c90, seq=3/768, ttl=64 (no response found!)
```

Exp3 - Pacotes ARP e ICMP, capturados no Tux124 durante o estabelecimento da ligação entre o Tux123 e o Tux122



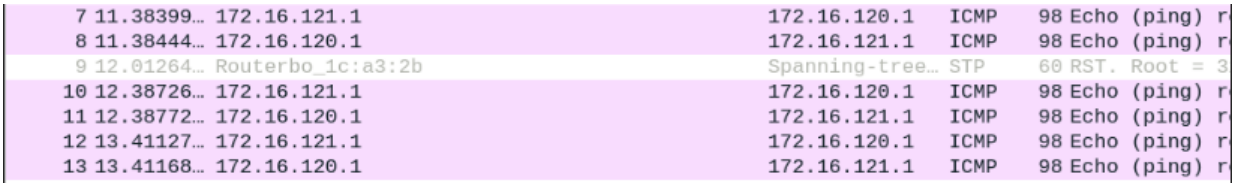
```
20 12.686971449  ec:75:0c:c2:3c:ac  Broadcast           ARP     60 Who has 172.16.120.254? Tell 172.16.120.1
21 12.687008322  ec:75:0c:c2:3c:75  ec:75:0c:c2:3c:ac   ARP     42 172.16.120.254 is at ec:75:0c:c2:3c:75
22 12.687157999  172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) request  id=0x50eb, seq=1/256, ttl=64 (reply in 23)
23 12.687634845  172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) reply    id=0x50eb, seq=1/256, ttl=63 (request in 22)
24 13.701706571  172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) request  id=0x50eb, seq=2/512, ttl=64 (reply in 25)
25 13.701956205  172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) reply    id=0x50eb, seq=2/512, ttl=63 (request in 24)
26 14.804528136  Routerbo_1c:a3:2d  Spanning-tree-(for-brid… STP  60 RST. Root = 32768/0/c4:ad:34:1c:a3:2c  Cost = 0  Port = 0x8002
```

Exp4 - Pacotes ARP e ICMP, capturados no Tux123 quando se remove a rota que liga o Tux122 ao Tux124



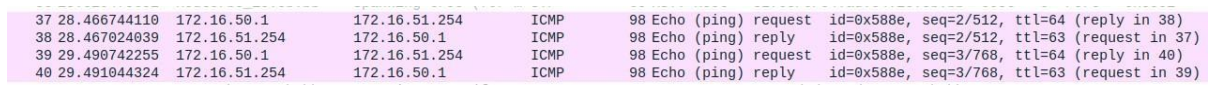
```
43 40.663370796  172.16.120.1      172.16.120.254      ICMP    98 Echo (ping) request  id=0x5175, seq=2/512, ttl=64 (reply in 44)
44 40.663560061  172.16.120.254    172.16.120.1        ICMP    98 Echo (ping) reply    id=0x5175, seq=2/512, ttl=64 (request in 43)
45 41.687394270  172.16.120.1      172.16.120.254      ICMP    98 Echo (ping) request  id=0x5175, seq=3/768, ttl=64 (reply in 46)
46 41.687633264  172.16.120.254    172.16.120.1        ICMP    98 Echo (ping) reply    id=0x5175, seq=3/768, ttl=64 (request in 45)
```

Exp4 - Pacotes ICMP transferidos entre Tux122 e o Router



```
 7 11.38399… 172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) r
 8 11.38444… 172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) r
 9 12.01264… Routerbo_1c:a3:2b Spanning-tree… STP     60 RST. Root = 3
10 12.38726… 172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) r
11 12.38772… 172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) r
12 13.41127… 172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) r
13 13.41168… 172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) r
```

Exp4 - Pacotes ICMP transferidos entre Tux52 e o Tux54



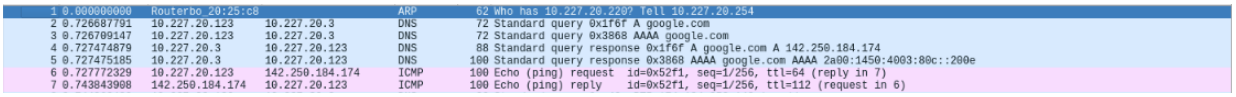
```
37 28.466744110  172.16.50.1       172.16.51.254       ICMP    98 Echo (ping) request  id=0x588e, seq=2/512, ttl=64 (reply in 38)
38 28.467024039  172.16.51.254     172.16.50.1         ICMP    98 Echo (ping) reply    id=0x588e, seq=2/512, ttl=63 (request in 37)
39 29.490742255  172.16.50.1       172.16.51.254       ICMP    98 Echo (ping) request  id=0x588e, seq=3/768, ttl=64 (reply in 40)
40 29.491044324  172.16.51.254     172.16.50.1         ICMP    98 Echo (ping) reply    id=0x588e, seq=3/768, ttl=63 (request in 39)
```

Exp4 - Pacotes ICMP de redirecionamento



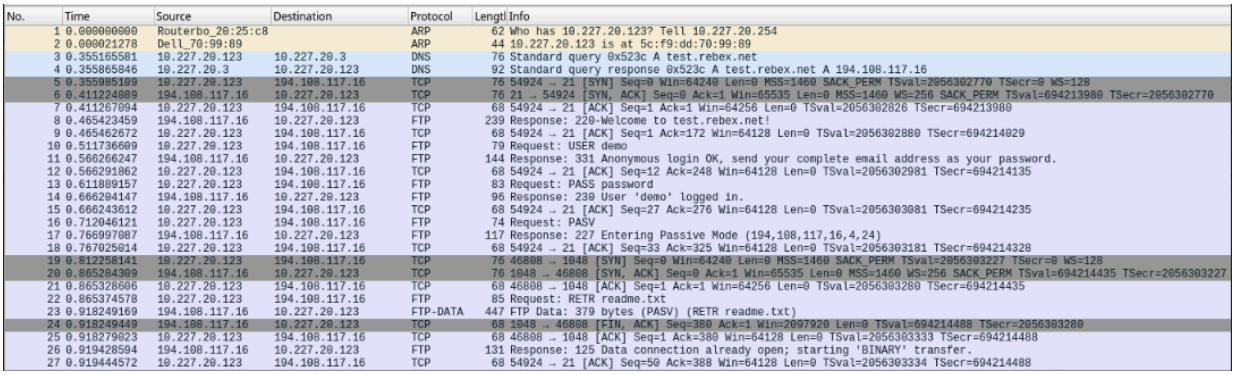
```
10 9.338634… 172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) r
11 9.338808… 172.16.121.254    172.16.121.1        ICMP   126 Redirect
12 9.339135… 172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) r
13 10.01046… Routerbo_1c:a3:2b Spanning-tree… STP     60 RST. Root = 3
14 10.36261… 172.16.121.1      172.16.120.1        ICMP    98 Echo (ping) r
15 10.36278… 172.16.121.254    172.16.121.1        ICMP   126 Redirect
16 10.36312… 172.16.120.1      172.16.121.1        ICMP    98 Echo (ping) r
```

Exp5 - Pacotes DNS no ping ao google.com



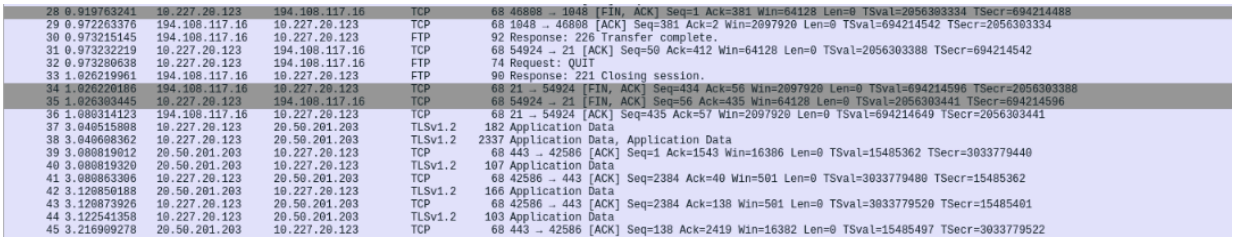
1	0.000000000	Routerbo_20:25:c8		ARP	62 Who has 10.227.20.220? Tell 10.227.20.254
2	0.726687791	10.227.20.123	10.227.20.3	DNS	72 Standard query 0x1f6f A google.com
3	0.726709147	10.227.20.123	10.227.20.3	DNS	72 Standard query 0x3868 AAAA google.com
4	0.727474879	10.227.20.3	10.227.20.123	DNS	88 Standard query response 0x1f6f A google.com A 142.250.184.174
5	0.727475185	10.227.20.3	10.227.20.123	DNS	100 Standard query response 0x3868 AAAA google.com AAAA 2a00:1450:4003:80c::200e
6	0.727772329	10.227.20.123	142.250.184.174	ICMP	100 Echo (ping) request id=0x52f1, seq=1/256, ttl=64 (reply in 7)
7	0.743843908	142.250.184.174	10.227.20.123	ICMP	100 Echo (ping) reply id=0x52f1, seq=1/256, ttl=112 (request in 6)

Exp6 - Início da transferência do ficheiro usando FTP



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Routerbo_20:25:c8		ARP	62	Who has 10.227.20.123? Tell 10.227.20.254
2	0.000021278	Dell_70:99:89		ARP	44	10.227.20.123 is at 5c:f9:dd:70:99:89
3	0.355165581	10.227.20.123	10.227.20.3	DNS	76	Standard query 0x523c A test.rebex.net
4	0.355865846	10.227.20.3	10.227.20.123	DNS	92	Standard query response 0x523c A test.rebex.net A 194.108.117.16
5	0.355985109	10.227.20.123	194.108.117.16	TCP	76	54924 → 21 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2056302770 TSecr=0 WS=128
6	0.411224089	194.108.117.16	10.227.20.123	TCP	76	21 → 54924 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=694213980 TSecr=2056302770
7	0.411267094	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2056302826 TSecr=694213980
8	0.465423459	194.108.117.16	10.227.20.123	FTP	239	Response: 220-Welcome to test.rebex.net!
9	0.465462672	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=1 Ack=172 Win=64128 Len=0 TSval=2056302880 TSecr=694214029
10	0.511736609	10.227.20.123	194.108.117.16	FTP	79	Request: USER demo
11	0.566266247	194.108.117.16	10.227.20.123	FTP	144	Response: 331 Anonymous login OK, send your complete email address as your password.
12	0.566291862	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=12 Ack=248 Win=64128 Len=0 TSval=2056302981 TSecr=694214135
13	0.611889157	10.227.20.123	194.108.117.16	FTP	83	Request: PASS password
14	0.666204147	194.108.117.16	10.227.20.123	FTP	96	Response: 230 User 'demo' logged in.
15	0.666243612	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=27 Ack=276 Win=64128 Len=0 TSval=2056303081 TSecr=694214235
16	0.712046121	10.227.20.123	194.108.117.16	FTP	74	Request: PASV
17	0.766997087	194.108.117.16	10.227.20.123	FTP	117	Response: 227 Entering Passive Mode (194,108,117,16,4,24)
18	0.767025014	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=33 Ack=325 Win=64128 Len=0 TSval=2056303181 TSecr=694214328
19	0.812258141	10.227.20.123	194.108.117.16	TCP	76	46808 → 1048 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2056303227 TSecr=0 WS=128
20	0.865284309	194.108.117.16	10.227.20.123	TCP	76	1048 → 46808 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=694214435 TSecr=2056303227
21	0.865328606	10.227.20.123	194.108.117.16	TCP	68	46808 → 1048 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2056303280 TSecr=694214435
22	0.865374578	10.227.20.123	194.108.117.16	FTP	85	Request: RETR readme.txt
23	0.918249169	194.108.117.16	10.227.20.123	FTP-DATA	447	FTP Data: 379 bytes (PASV) (RETR readme.txt)
24	0.918249449	194.108.117.16	10.227.20.123	TCP	68	1048 → 46808 [FIN, ACK] Seq=380 Ack=1 Win=2097920 Len=0 TSval=694214488 TSecr=2056303280
25	0.918279023	10.227.20.123	194.108.117.16	TCP	68	46808 → 1048 [ACK] Seq=1 Ack=380 Win=64128 Len=0 TSval=2056303333 TSecr=694214488
26	0.919428594	194.108.117.16	10.227.20.123	FTP	131	Response: 125 Data connection already open; starting 'BINARY' transfer.
27	0.919444572	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=50 Ack=388 Win=64128 Len=0 TSval=2056303334 TSecr=694214488

Exp6 - Fim da transferência do ficheiro usando FTP



28	0.919763241	10.227.20.123	194.108.117.16	TCP	68	46808 → 1048 [FIN, ACK] Seq=1 Ack=381 Win=64128 Len=0 TSval=2056303334 TSecr=694214488
29	0.972263376	194.108.117.16	10.227.20.123	TCP	68	1048 → 46808 [ACK] Seq=381 Ack=2 Win=2097920 Len=0 TSval=694214542 TSecr=2056303334
30	0.973215145	194.108.117.16	10.227.20.123	FTP	92	Response: 226 Transfer complete.
31	0.973232219	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [ACK] Seq=50 Ack=412 Win=64128 Len=0 TSval=2056303388 TSecr=694214542
32	0.973280638	10.227.20.123	194.108.117.16	FTP	74	Request: QUIT
33	1.026219961	194.108.117.16	10.227.20.123	FTP	90	Response: 221 Closing session.
34	1.026220186	194.108.117.16	10.227.20.123	TCP	68	21 → 54924 [FIN, ACK] Seq=434 Ack=56 Win=2097920 Len=0 TSval=694214596 TSecr=2056303388
35	1.026303445	10.227.20.123	194.108.117.16	TCP	68	54924 → 21 [FIN, ACK] Seq=56 Ack=435 Win=64128 Len=0 TSval=2056303441 TSecr=694214596
36	1.080314123	194.108.117.16	10.227.20.123	TCP	68	21 → 54924 [ACK] Seq=435 Ack=57 Win=2097920 Len=0 TSval=694214649 TSecr=2056303441
37	3.040515808	10.227.20.123	20.50.201.203	TLSv1.2	182	Application Data
38	3.040608362	10.227.20.123	20.50.201.203	TLSv1.2	2337	Application Data, Application Data
39	3.080819012	20.50.201.203	10.227.20.123	TCP	68	443 → 42586 [ACK] Seq=1 Ack=1543 Win=16386 Len=0 TSval=15485362 TSecr=3033779440
40	3.080819320	20.50.201.203	10.227.20.123	TLSv1.2	107	Application Data
41	3.080863306	10.227.20.123	20.50.201.203	TCP	68	42586 → 443 [ACK] Seq=2384 Ack=40 Win=501 Len=0 TSval=3033779480 TSecr=15485362
42	3.120850188	20.50.201.203	10.227.20.123	TLSv1.2	166	Application Data
43	3.120873926	10.227.20.123	20.50.201.203	TCP	68	42586 → 443 [ACK] Seq=2384 Ack=138 Win=501 Len=0 TSval=3033779520 TSecr=15485401
44	3.122541358	10.227.20.123	20.50.201.203	TLSv1.2	103	Application Data
45	3.216909278	20.50.201.203	10.227.20.123	TCP	68	443 → 42586 [ACK] Seq=138 Ack=2419 Win=16382 Len=0 TSval=15485497 TSecr=3033779522